

Document number:	P0547R2
Date:	2017-07-11
Project:	C++ Extensions for Ranges
Reply-to:	Eric Niebler < eniebler@boost.org >
Audience:	Library Working Group

Ranges TS: Assorted Object Concept Fixes

- [1 Synopsis](#)
- [2 Revision History](#)
 - [2.1 R1](#)
 - [2.2 R2](#)
- [3 Problem description](#)
- [4 Solution description](#)
- [5 Proposed Resolution](#)
- [6 Acknowledgements](#)

1 Synopsis

This paper suggests reformulations of the following fundamental object concepts to resolve a number of outstanding issues, and to bring them in line with (what experience has shown to be) user expectations:

- Destructible
- Constructible
- DefaultConstructible
- MoveConstructible
- CopyConstructible
- Assignable

The suggested changes make them behave more like what their associated type traits do.

In addition, we suggest a change to `Movable` that correctly positions it as the base of the `Regular` concept hierarchy, which concerns itself with types with value semantics.

2 Revision History

2.1 R1

- [Per LWG Kona direction, removed the requirement that `Destructible` types do not overload `operator&`.](#)
- [Add `Readable` and `Writable` changes \(Issue #330, #381, #387, #399\).](#)
- [Reorganize bullet points in `Move/CopyConstructible`.](#)
- [Add missing comma in the new description of `Destructible`.](#)
- [Rebase `Assignable` changes onto the post-P0370 wording.](#)
- [Address #293 in `MoveConstructible`.](#)
- [s/there is no subsumption relationship/there need not be any subsumption relationship/g.](#)

2.2 R2

- Add this revision history
- Concept definitions in the style of “`Concept<T>()` is satisfied if and only if:” should use “only if.”

3 Problem description

The [Palo Alto report](#), on which the design of the Ranges TS is based, suggested the object concepts `Semiregular` and `Regular` for constraining standard library components. In an appendix it concedes that many generic components could more usefully be constrained with decompositions of these very coarse concepts: `Movable` and `Copyable`.

While implementing a subset of a constrained Standard Library, the authors of the Ranges TS found that even more fine-grained

“object” concepts were often useful: `MoveConstructible`, `CopyConstructible`, `Assignable`, and others. These concepts are needed to avoid over-constraining low-level library utilities like `pair`, `tuple`, `variant` and more. Rather than aping the similarly named type-traits, the authors of the Ranges TS tried to preserve the intent of the Palo Alto report by giving them semantic weight. It did this in various ways, including:

- Requiring that destructible objects can have their address taken.
- Requiring that destructible objects can have their destructor called explicitly.
- Requiring that constructible objects are destructible.
- Testing that objects could be allocated and deallocated in dynamic as well as automatic storage.
- Testing for array allocation and deallocation.
- Testing that default constructors are not explicit.

Although well-intentioned, many of the extra semantic requirements have proved to be problematic in practice. Here, for instance, are seven currently open [stl2](#) bugs that need resolution:

1. **“Why do neither reference types nor array types satisfy `Destructible`?”** ([stl2#70](#))

This issue, raised independently by Walter Brown, Alisdair Meredith, and others, questions the decision to have `Destructible<T>()` require the valid expression `t.~T()` for some lvalue `t` of type `T`. That has the effect of preventing reference and array types from satisfying `Destructible`.

In addition, `Destructible` requires `&t` to have type `T*`. This also prevents reference types from satisfying `Destructible` since you can’t form a pointer to a reference.

A reasonable interpretation of “destructible” is “can fall out of scope”. This is roughly what is tested by the `is_destructible` type trait. By this rubric, references and array types should satisfy `Destructible` as they do for the trait.

2. **“Is it intended that `Constructible<int&, long&>()` is true?”** ([stl2#301](#))

`Constructible<T, Args...>()` tries to test that the type `T` can be constructed on the heap as well as in automatic storage. But requiring the expression `new T{declval<Args>()...}` causes reference types to fail to satisfy the concept since references cannot be dynamically allocated. `Constructible` “solves” this problem by handling references separately; their required expression is merely `T(declval<Args>()...)`. That syntax has the unfortunate effect of being a function-style cast, which in the case of `int&` and `long&`, amounts to a `reinterpret_cast`.

We could patch this up by using universal initialization syntax, but that comes with its own problems. Instead, we opted for a more radical simplification: just do what `is_constructible` does.

3. **“`Movable<int&&>()` is true and it should probably be false”** ([stl2#310](#))

A cursory review of the places that use `Movable` in the Ranges TS reveals that they all are expecting types with value semantics. A reference does not exhibit value semantics, so it is surprising for `int&&` to satisfy `Movable`.

4. **“Is it intended that an aggregate with a deleted or nonexistent default constructor satisfy `DefaultConstructible`?”** ([stl2#300](#))

Consider a type such as the following:

```
struct A{
    A(const A&) = default;
};
```

This type is not default constructible; the statement `auto a = A();` is ill-formed. However, since `A` is an aggregate, the statement `auto a = A{};` is well-formed. Since `Constructible` is testing for the latter syntax and not the former, `A` satisfies `DefaultConstructible`. This is in contrast with the result of `std::is_default_constructible<A>::value`, which is false.

5. **“`Assignable` concept looks wrong”** ([stl2#229](#))

There are a few problems with `Assignable`. The given definition, `Assignable<T, U>()` would appear to work with reference types (as one would expect), but the prose description reads, “Let `t` be an lvalue of type `T...`” There are no lvalues of reference type, so the wording is simply wrong. The wording also erroneously uses `==` instead of the magic phrase “is equal to,” accidentally requiring the types to satisfy (some part of) `EqualityComparable`.

Also, LWG requested at the Issaquah 2016 meeting that this concept be changed such that it is only satisfied

when τ is an lvalue reference type.

6. “MoveConstructible() != std::is_move_constructible()” ([stl2#313](#))

The definition of MoveConstructible applies `remove_cv_t` to its argument before testing it, as shown below:

```
template <class T>
concept bool MoveConstructible() {
    return Constructible<T, remove_cv_t<T>&&>() &&
        ConvertibleTo<remove_cv_t<T>&&, T>();
}
```

This somewhat surprisingly causes `const some_move_only_type` to satisfy MoveConstructible, when it probably shouldn't. `std::is_move_constructible<const some_move_only_type>::value` is false, for instance.

7. “Subsumption and object concepts” ([CaseyCarter/stl2#22](#))

This issue relates to the fact that there is *almost* a perfect sequence of subsumption relationships from Destructible, through Constructible, and all the way to Regular. The “almost” is the problem. Given a set of overloads constrained with these concepts, there will be ambiguity due to the fact that *in some cases* Constructible does not subsume Destructible (e.g., for references).

We were also motivated by the very real user confusion about why concepts with names similar to associated type traits gives different answers for different types and type categories.

It remains our intention to resist the temptation to constrain the library with semantically meaningless, purely syntactic concepts.

4 Solution description

At the high level, the solution this paper suggests is to break the object concepts into two logical groups: the lower-level concepts that follow the lead of their similarly-named type traits with regard to “odd” types (references, arrays, cv void), and the higher-level concepts that deal only with value semantic types.

The lower-level concepts are those that have corresponding type traits, and behave largely like them. They can no longer properly be thought of as “object” concepts, so they rightly belong with the core language concepts.

- Destructible
- Constructible
- DefaultConstructible
- MoveConstructible
- CopyConstructible
- Assignable

These concepts are great for constraining the special members of low-level generic facilities like `std::tuple` and `std::optional`, but they are too fiddly for constraining anything but the most trivial generic algorithms. Unlike the type traits, these concepts require additional syntax and semantics for the sake of the generic programmer's sanity, although the requirements are light.

The higher-level concepts are those that the Palo Alto report describes, and are satisfied by object types only:

- Movable
- Copyable
- Semiregular
- Regular

These are the concepts that largely constrain the algorithms in the STL.

The changes suggested in this paper bear on [LWG#2146](#), “Are reference types Copy/Move-Constructible/Assignable or Destructible?” There seems to be some discomfort with the current behavior of the type traits with regard to reference types. Should that issue be resolved such that reference types are deemed to *not* be copy/move-constructible/assignable or destructible, the concepts should follow suit. Until such time, the authors feel that hewing to the behavior of the traits is the best way to avoid confusion.

In the “Proposed Resolution” that follows, there are editorial notes that highlight specific changes and describe their intent and impact.

5 Proposed Resolution

[Editor's note: Edit subsection "Concept Assignable" ([concepts.lib.corelang.assignable]) as follows:]

```
template <class T, class U>
concept bool Assignable() {
    return CommonReference<const T&, const U&>() && requires(T&& t, U&& u) {
    { std::forward<T>(t) = std::forward<U>(u) } -> Same<T&>;
};
return is_lvalue_reference<T>::value && // see below
CommonReference<
    const remove_reference_t<T>&,
    const remove_reference_t<U>&>() &&
requires(T t, U&& u) {
    { t = std::forward<U>(u) } -> Same<T>&&;
}
}
```

1 ~~Let t be an lvalue of type T , and r be the type $\text{remove_reference_t}<U>$. If U is an lvalue reference type, let v be an lvalue of type R ; otherwise, let v be an rvalue of type R . Let uu be a distinct object of type R such that uu is equal to v . Let t be an lvalue which refers to an object o such that $\text{decltype}((t))$ is T , and u an expression such that $\text{decltype}((u))$ is U . Let $u2$ be a distinct object that is equal to u . Then $\text{Assignable}<T, U>()$ is satisfied only if~~

(1.1) – $\text{addressof}(t = \text{v}u) == \text{addressof}(\text{t}o)$.

(1.2) – After evaluating $t = \text{v}u$, t is equal to $uuu2$ and:

(1.2.1) – If $\text{v}u$ is a non-const ~~rvalue, its~~xvalue, the resulting state of the object to which it refers is valid but unspecified ([lib.types.movedfrom]).

(1.2.2) – Otherwise, ~~if u is a glvalue, the object to which it refers~~ is not modified.

2 There need not be any subsumption relationship between $\text{Assignable}<T, U>()$ and $\text{is_lvalue_reference}<T>::\text{value}$.

[Editor's note: Prior to this change, Assignable is trying to work with proxy reference types and failing. It perfectly forwards its arguments, but requires the return type of assignment to be $T\&$ (which is not true for some proxy types). Also, the allowable moved-from state of the rhs expression (u) is described in terms of its value category. But if the rhs is a proxy reference (e.g., $\text{reference_wrapper}<\text{int}>$) then the value category of the proxy bears no relation to the value category of the referent.

The issue was discussed in the Issaquah 2016 meeting. The guidance given there was to narrowly focus this concept on "traditional" assignability only – assignments to non-const lvalues from non-proxy expressions – and solve the proxy problem at a later date. That is the direction taken here.]

[Editor's note: Move subsection "Concept Destructible" ([concepts.lib.object.destructible]) to subsection "Core language concepts" ([concepts.lib.corelang]) after [concepts.lib.corelang.swappable], change its stable id to [concepts.lib.corelang.destructible] and edit it as follows:]

1 ~~The Destructible concept is the base of the hierarchy of object concepts. It specifies properties that all such object types have in common. The Destructible concept specifies properties of all types, instances of which can be destroyed at the end of their lifetime, or reference types.~~

```
template <class T>
concept bool Destructible() {
    return requires(T t, const T ct, T* p) {
        { t.~T() } noexcept;
        { &t } -> Same<T*>; // not required to be equality preserving
        { &ct } -> Same<const T*>; // not required to be equality preserving
        delete p;
        delete[] p;
};
return is_nothrow_destructible<T>::value; // see below
}
```

~~2 The expression requirement $\&t$ does not require implicit expression variants.~~

~~3 Given a (possibly const) lvalue t of type T and pointer p of type T^* , $\text{Destructible}<T>()$ is satisfied only if~~

(3.1) – ~~After evaluating the expression $t.~T()$, $\text{delete } p$, or $\text{delete}[] p$, all resources owned by the denoted object(s) are reclaimed.~~

(3.2) – ~~$\&t == \text{addressof}(t)$.~~

(3.3) — ~~The expression &t is non-modifying.~~

2 There need not be any subsumption relationship between `Destructible<T>()` and `is_nothrow_destructible<T>::value`.

[Editor's note: In the minutes of Ranges TS wording review at Kona on 2015-08-14, the following is recorded:

In 19.4.1 Alisdair asks whether reference types are Destructible. Eric pointed to [issue 70](#), regarding reference types and array types. Alisdair concerned that Destructible sounds like something that goes out of scope, maybe this concept is really describing Deletable.

We took this as guidance to make Destructible behave more like the type traits with regard to “strange” types like references and arrays. We also dropped the requirement for dynamic [array] deallocation. Per discussion in Kona 2016, we drop the requirement for a sane address-of operation. We require that destructors are marked noexcept since noexcept clauses throughout the standard and the Ranges TS tacitly assume it, and because sane implementations require it.]

[Editor's note: Move subsection “Concept Constructible” ([`concepts.lib.object.constructible`]) to subsection “Core language concepts” ([`concepts.lib.corelang`]) after [`concepts.lib.corelang.destructible`], change its stable id to [`concepts.lib.corelang.constructible`] and edit it as follows:]

1 The Constructible concept is used to constrain the ~~type of a variable to be either an object type constructible from initialization of a variable of a type with~~ a given set of argument types, ~~or a reference type that can be bound to those arguments.~~

```
template <class T, class... Args>
concept bool __ConstructibleObject = // exposition-only
    Destructible<T>() && requires(Args&&... args) {
        T{std::forward<Args>(args)...}; // not required to be equality preserving
        new T{std::forward<Args>(args)...}; // not required to be equality preserving
    };

template <class T, class... Args>
concept bool __BindableReference = // exposition-only
    is_reference<T>::value && requires(Args&&... args) {
        T{std::forward<Args>(args)...};
    };

template <class T, class... Args>
concept bool Constructible() {
    return __ConstructibleObject<T, Args...> ||
           __BindableReference<T, Args...>;
    return Destructible<T>() && is_constructible<T, Args...>::value; // see below
}
```

2 There need not be any subsumption relationship between `Constructible<T, Args...>()` and `is_constructible<T, Args...>::value`.

[Editor's note: Constructible now always subsumes Destructible, fixing [CaseyCarter/stl2#22](#) which regards overload ambiguities introduced by the lack of such a simple subsumption relationship. Constructible follows Destructible by dropping the requirement for dynamic [array] allocation.]

[Editor's note: Move subsection “Concept DefaultConstructible” ([`concepts.lib.object.defaultconstructible`]) to subsection “Core language concepts” ([`concepts.lib.corelang`]) after [`concepts.lib.corelang.constructible`], change its stable id to [`concepts.lib.corelang.defaultconstructible`] and edit it as follows:]

```
template <class T>
concept bool DefaultConstructible() {
    return Constructible<T>() &&
        requires(const-size_t n) {
            new T[n]{}; // not required to be equality preserving
        };
}
```

1 [Note: The array allocation expression `new T[n]{}` implicitly requires that `T` has a non-explicit default constructor. —end note]

[Editor's note: `DefaultConstructible<T>()` could trivially be replaced with `Constructible<T>()`. We are ambivalent about whether to remove `DefaultConstructible` or not, although we note that keeping it gives us the opportunity to augment this concept to require non-explicit default constructibility. Such a requirement is trivial to add, should the committee decide to.]

[Editor's note: Move subsection “Concept MoveConstructible” ([`concepts.lib.object.moveconstructible`]) to subsection “Core language concepts” ([`concepts.lib.corelang`]) after [`concepts.lib.corelang.defaultconstructible`], change its stable id to

[[concepts.lib.corelang.moveconstructible](#)] and edit it as follows:]

```
template <class T>
concept bool MoveConstructible() {
    return Constructible<T, remove_cv_t<T>&&>() &&
        ConvertibleTo<remove_cv_t<T>&&, T>();
}
```

1 If τ is an object type, then let rv be an rvalue of type ~~$\text{remove_cv_t}<\tau>$~~ and u a distinct object of type τ equal to rv . Then $\text{MoveConstructible}<\tau>()$ is satisfied only if

(1.1) – After the definition $\tau \ u = rv;$, u is equal to ~~u~~ u ~~the value of rv before the construction.~~

(1.2) – $\tau\{rv\}$ ~~or $*\text{new } \tau\{rv\}$~~ is equal to u ~~the value of rv before the construction.~~

(1.32) – If τ is not const, rv 's resulting state is valid but unspecified ([lib.types.movedfrom]); otherwise, it is unchanged.

[Editor's note: We no longer strip top-level `const` from the parameter to harmonize `MoveConstructible` with `is_move_constructible`. And as with `is_move_constructible`, `MoveConstructible<int&&>()` is true. See [LWG#2146](#).

The description of `MoveConstructible` adds semantic requirements when τ is an object type. It says nothing about non-object types because no additional semantic requirements are necessary.]

[Editor's note: Move subsection "Concept `CopyConstructible`" ([[concepts.lib.object.copyconstructible](#)]) to subsection "Core language concepts" ([[concepts.lib.corelang](#)]) after [[concepts.lib.corelang.moveconstructible](#)], change its stable id to [[concepts.lib.corelang.copyconstructible](#)] and edit it as follows:]

```
template <class T>
concept bool CopyConstructible() {
    return MoveConstructible<T>() &&
        Constructible<T, const remove_cv_t<T>&&>() &&
        ConvertibleTo<remove_cv_t<T>&, T>() &&
        ConvertibleTo<const remove_cv_t<T>&, T>() &&
        ConvertibleTo<const remove_cv_t<T>&&, T>();
        Constructible<T, T&>() && ConvertibleTo<T&, T>() &&
        Constructible<T, const T&>() && ConvertibleTo<const T&, T>() &&
        Constructible<T, const T>() && ConvertibleTo<const T, T>();
}
```

1 If τ is an object type, then let v be an lvalue of type (possibly `const`) ~~$\text{remove_cv_t}<\tau>$~~ or an rvalue of type `const  $\text{remove\_cv\_t}<\tau>$` . Then $\text{CopyConstructible}<\tau>()$ is satisfied only if

(1.1) – After the definition $\tau \ u = v;$, u is equal to v .

(1.2) – $\tau\{v\}$ ~~or $*\text{new } \tau\{v\}$~~ is equal to v .

[Editor's note: As with `MoveConstructible`, we no longer strip top-level `cv`-qualifiers to bring `CopyConstructible` into harmony with `is_copy_constructible`.

Since `Constructible` no longer directly tests that $\tau(\text{args} \dots)$ is a valid expression, it doesn't implicitly require the `cv`-qualified expression variants as described in subsection "Equality Preservation" ([[concepts.lib.general.equality](#)]/6). As a result, we needed to *explicitly* add the additional requirements for `Constructible<T, T&>()` and `Constructible<T, const T&&>()`.

Like `MoveConstructible`, `CopyConstructible` adds no additional semantic requirements for non-object types.]

[Editor's note: Edit subsection "Concept `Movable`" ([[concepts.lib.object.movable](#)]) as follows:]

```
template <class T>
concept bool Movable() {
    return is_object<T>::value && MoveConstructible<T>() &&
        Assignable<T&, T>() &&
        Swappable<T&>();
}
```

1 There need not be any subsumption relationship between `Movable<T>()` and `is_object<T>::value`.

[Editor's note: `Movable` is the base concept of the `Regular` hierarchy. These concepts are concerned with value semantics. As such, it makes no sense for `Movable<int&&>()` to return `true` ([stl2#310](#)). We add the requirement that τ is an object type to resolve the issue. Since `Movable` is subsumed by `Copyable`, `Semiregular`, and `Regular`, these concepts will only ever be satisfied by object types.]

[Editor's note: Edit subsection "Concept `Readable`" ([[iterators.readable](#)]) as follows (also includes the fix for [stl2#330](#) and

[stl2#399](#));]

```
template <class In>
concept bool Readable() {
    return Movable<In>() & DefaultConstructible<In>() &&
    return requires(const In& i) {
        typename value_type_t<In>;
        typename reference_t<In>;
        typename rvalue_reference_t<In>;
        {*i}--> Same<reference_t<In>>;
        { ranges::iter_move(i) }--> Same<rvalue_reference_t<In>>;
    } &&
    CommonReference<reference_t<In>&&, value_type_t<In>&>() &&
    CommonReference<reference_t<In>&&, rvalue_reference_t<In>&&() &&
    CommonReference<rvalue_reference_t<In>&&, const value_type_t<In>&>();
}
```

[Editor's note: Edit subsection "Concept Writable" ([iterators.writable]) as follows (also includes the fixes for [stl2#381](#) and [stl2#387](#));]

```
template <class Out, class T>
concept bool Writable() {
    return Movable<Out>() & DefaultConstructible<Out>() &&
    requires(Out&& o, T&& t) {
        *o = std::forward<T>(t); // not required to be equality preserving
        *std::forward<Out>(o) = std::forward<T>(t); // not required to be equality preserving
        const_cast<const reference_t<Out>&&>(*o) =
        std::forward<T>(t); // not required to be equality preserving
        const_cast<const reference_t<Out>&&>(*std::forward<Out>(o)) =
        std::forward<T>(t); // not required to be equality preserving
    };
}
```

6 Acknowledgements

I would like to thank Casey Carter and Andrew Sutton for their review feedback.